Aprendizaje de Máquina
Sesión 21: Aprendizaje de máquina para textos

Angel Rodríguez

Práctica #7: Clasificación de documentos

Entrega: Jueves 22 de Octubre.

Nuevo experimento

Repetir la práctica de clasificación de documentos de la clase anterior (clase #20), sustituyendo TF-IDF por una representación basada en frecuencias de palabras.

1. Cargar los documentos y sus etiquetas.
2. Separar los datos en entrenamiento y prueba.
3. Construir un `Pipeline` con `TfidfVectorizer` y `LogisticRegression`.
4. Entrenar el modelo y evaluar su precisión en el conjunto de prueba.
5. Examinar predicciones correctas e incorrectas.
6. Identificar las palabras con mayor influencia en cada categoría.
7. **Buscar una noticia en Internet** e intentar predecir su categoría utilizando ambos modelos (TF-IDF y CountVectorizer).

Matriz de confusión

Para clasificación binaria comparamos la predicción del sistema con la etiqueta correcta o *gold label*.

		Etiquetas reales		
		Positivo	**Negativo**	
Predicción	**Positivo**	**TP**	**FP**	Precision $= \frac{TP}{TP + FP}$
	Negativo	**FN**	**TN**	
		Recall $= \frac{TP}{TP + FN}$		Accuracy $= \frac{TP + TN}{TP + FP + TN + FN}$

donde:

TP : verdaderos positivos,
FP : falsos positivos,
FN : falsos negativos,
TN : verdaderos negativos.

F1-score

Podemos combinar Precision y Recall mediante la media armónica:

$$\boxed{F_1 = 2\frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}}$$

El valor de F_1 será alto únicamente si

$$\boxed{\text{Precision y Recall son ambas altas.}}$$

¿Qué ocurre con varias clases?

Supongamos una clasificación de correos:

$$y \in \{\text{urgente}, \text{normal}, \text{spam}\}.$$

Podemos calcular Precision y Recall **por separado para cada clase**:

$$P_{\text{urgente}}, \quad P_{\text{normal}}, \quad P_{\text{spam}},$$

$$R_{\text{urgente}}, \quad R_{\text{normal}}, \quad R_{\text{spam}}.$$

Pero también queremos una única métrica global.

Ejemplo: matriz de confusión multiclase

	Categoría real			
Predicción	**Urgente**	**Normal**	**Spam**	**Precisión**
Urgente	8	10	1	$\frac{8}{8+10+1}$
Normal	5	60	50	$\frac{60}{5+60+50}$
Spam	3	30	200	$\frac{200}{3+30+200}$
Exhaustividad	$\frac{8}{8+5+3}$	$\frac{60}{10+60+30}$	$\frac{200}{1+50+200}$	

- **Azul:** predicciones correctas (diagonal).
- **Rojo:** predicciones incorrectas.

Macro-average

En **macro-averaging** calculamos primero la métrica para cada clase y después promediamos.
Por ejemplo:

$$P_{\text{macro}} = \frac{1}{K} \sum_{k=1}^{K} P_k$$

y de forma análoga para Recall y F_1.

Cada clase contribuye

con el mismo peso.

Micro-average

En **micro-averaging** primero agregamos las decisiones de todas las clases:

$$TP_{\text{total}} = \sum_k TP_k,$$

$$FP_{\text{total}} = \sum_k FP_k.$$

Después calculamos

$$\boxed{P_{\text{micro}} = \frac{TP_{\text{total}}}{TP_{\text{total}} + FP_{\text{total}}}}$$

Las clases frecuentes tienen mayor influencia en el resultado.

Precisión macro y micro en clasificación multiclase

Clase 1: Urgente

	Real +	Real –
Pred. +	8	11
Pred. –	8	340

Clase 2: Normal

	Real +	Real –
Pred. +	60	55
Pred. –	40	212

Clase 3: Spam

	Real +	Real –
Pred. +	200	33
Pred. –	51	83

Global (pooled)

	Real +	Real –
Pred. +	268	99
Pred. –	99	635

$$P_{\text{urgente}} = \frac{8}{8+11} \approx 0{,}42, \qquad P_{\text{normal}} = \frac{60}{60+55} \approx 0{,}52,$$

$$P_{\text{spam}} = \frac{200}{200+33} \approx 0{,}86.$$

Promedios de precisión

$$P_{\text{macro}} = \frac{P_{\text{urgente}} + P_{\text{normal}} + P_{\text{spam}}}{3} \approx 0{,}60$$

$$P_{\text{micro}} = \frac{TP_{\text{total}}}{TP_{\text{total}} + FP_{\text{total}}} = \frac{268}{268+99} \approx 0{,}73$$

Micro vs. Macro

	Micro	**Macro**
Primero	Agrupa ejemplos	Evalúa cada clase
Después	Calcula métrica	Promedia métricas
Peso	Más a clases grandes	Igual por clase

Cuando todas las clases son igualmente importantes,

Macro-average

permite observar mejor el desempeño sobre las clases pequeñas.

⚠¡Importante!⚠Cuidado con el *data leakage*

Supongamos que queremos utilizar TF-IDF.

No debemos hacer:

fit TF-IDF con todos los documentos → cross-validation

porque el vocabulario y los pesos IDF ya contienen información de los folds de validación.

El vectorizador debe ajustarse

dentro de cada fold de entrenamiento.

Validación cruzada: ¿qué debe suceder en cada fold?

En validación cruzada con $k = 5$ por ejemplo, utilizamos el 80 % de los datos para entrenamiento y el 20 % para validación.

Paso	**Entrenamiento (80 %)**	**Validación (20 %)**
1. TF-IDF	`fit_transform()`	`transform()`
2. Clasificador	`fit()`	`predict()`
3. Evaluación	—	F1, accuracy, etc.

Primer fold:

$$\underbrace{D_1, D_2, D_3, D_4}_{\substack{\text{Entrenamiento} \\ \text{ajustar TF-IDF}}} \qquad \underbrace{D_5}_{\substack{\text{Validación} \\ \text{solo transformar}}}$$

Segundo fold:

$$\underbrace{D_1, D_2, D_3, D_5}_{\substack{\text{Entrenamiento} \\ \text{ajustar nuevo TF-IDF}}} \qquad \underbrace{D_4}_{\substack{\text{Validación} \\ \text{solo transformar}}}$$

Importante

El vectorizador TF-IDF se ajusta nuevamente en cada fold, sin utilizar los documentos de validación.

Pipeline y validación cruzada

Scikit-Learn permite evaluar todo el proceso:

```
from sklearn.pipeline import Pipeline
from sklearn.model_selection import cross_val_score
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

model = Pipeline([
    ("tfidf", TfidfVectorizer()),
    ("clf", LogisticRegression())
])

scores = cross_val_score(
    model,
    X_train,
    y_train,
    cv=5,
    scoring="f1_macro"
)
```

¿Por qué usar un Pipeline?

En cada fold, Scikit-Learn ejecuta:

fit del vectorizador → transformación → fit del clasificador

utilizando únicamente los datos de entrenamiento de ese fold.

Así evitamos que

información de validación → se filtre al entrenamiento.

Comparando representaciones

Podemos comparar, por ejemplo,

$$\texttt{CountVectorizer} + \texttt{LogisticRegression}$$

contra

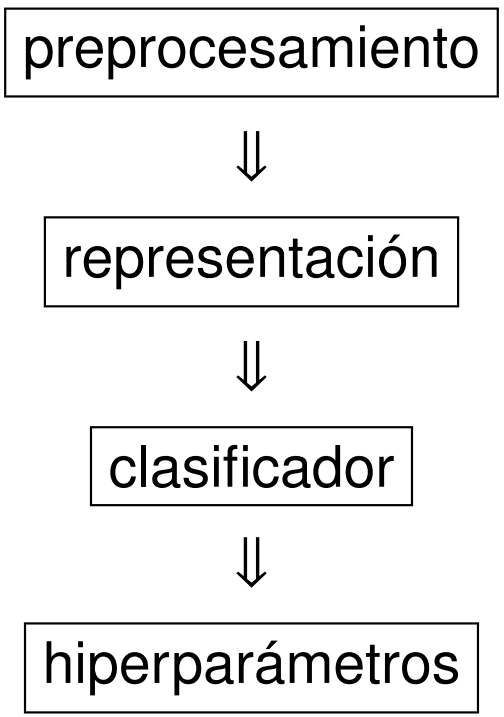
$$\texttt{TfidfVectorizer} + \texttt{LogisticRegression}.$$

La comparación debe realizarse

sobre los mismos folds y con la misma métrica.

¿Qué estamos seleccionando realmente?

Un modelo de texto incluye varias decisiones:



Por tanto, debemos comparar



Selección de hiperparámetros

Por ejemplo, podemos modificar:

- uso de unigramas o bigramas (es decir, considerar dos palabras como “una” sola);
- `min_df`;
- `max_features`;
- tipo de regularización;
- intensidad de regularización.

Cada combinación define un modelo diferente:

$$\theta_{\text{hiper}} \longrightarrow \text{desempeño en validación}$$

Búsqueda de hiperparámetros

Podemos utilizar[1]:

```
from sklearn.model_selection import GridSearchCV

params = {
    "tfidf__ngram_range": [(1,1), (1,2)],
    "clf__C": [0.1, 1, 10]
}

search = GridSearchCV(
    model,
    params,
    cv=5,
    scoring="f1_macro"
)

search.fit(X_train, y_train)
```

[1] Recordemos que podemos ver los parámetros con `print(model.get_params())`

El parámetro C en Scikit-Learn

En `LogisticRegression`, Scikit-Learn utiliza

$$C$$

como el inverso de la fuerza de regularización.
Por tanto:

$$\boxed{C \downarrow \Rightarrow \text{más regularización}}$$

$$\boxed{C \uparrow \Rightarrow \text{menos regularización.}}$$

Por eso podemos incluir C entre los hiperparámetros seleccionados mediante validación cruzada.

4.11 Pruebas de significancia estadística

Al desarrollar sistemas de NLP, frecuentemente necesitamos comparar dos clasificadores.

Por ejemplo:

- ¿Nuestro nuevo clasificador es mejor que el anterior?
- ¿La mejora en F1 es suficientemente convincente?
- ¿Obtendríamos una mejora similar con otro test set?

Problema

Una diferencia observada entre dos modelos puede deberse, al menos parcialmente, a las características particulares del conjunto de prueba.

La **significancia estadística** nos ayuda a evaluar qué tan sorprendente sería la diferencia observada bajo una hipótesis nula.

Comparación de dos clasificadores

Sean A y B dos clasificadores y sea x un conjunto de prueba. Definimos $M(A, x)$ como el desempeño del modelo A sobre x, utilizando una métrica M.

La diferencia de desempeño es:

$$\boxed{\delta(x) = M(A, x) - M(B, x)} \tag{4.45}$$

- $\delta(x) > 0$: el modelo A obtiene mejor resultado sobre x.
- $\delta(x) = 0$: ambos obtienen el mismo resultado.
- $\delta(x) < 0$: el modelo B obtiene mejor resultado.

Tamaño del efecto

$\delta(x)$ también es llamado *effect size*, y cuantifica la magnitud de la diferencia observada.

¿Es suficiente observar una mejora?

Supongamos que comparamos dos clasificadores de sentimientos:

$$M(A, x) = 0{,}88, \qquad M(B, x) = 0{,}84.$$

Por tanto,

$$\delta(x) = 0{,}88 - 0{,}84 = \boxed{0{,}04}.$$

¿Podemos concluir que A es mejor que B?

No necesariamente.

- El conjunto x es solamente una muestra.
- Otro conjunto de prueba podría producir una diferencia distinta.
- Necesitamos cuantificar la evidencia estadística de la mejora.

Pregunta central

¿Sería sorprendente observar una diferencia de 0,04 si A no fuera mejor que B?

Hipótesis nula y alternativa

Se puede plantear una prueba unilateral:

$$\boxed{\begin{aligned} H_0 &: \delta(x) \leq 0, \\ H_1 &: \delta(x) > 0. \end{aligned}} \tag{4.46}$$

Hipótesis nula H_0:
El clasificador A no es mejor que B.

Hipótesis alternativa H_1:
El clasificador A es mejor que B.

Objetivo

Buscar evidencia suficiente para rechazar H_0 y apoyar la afirmación de que A tiene mejor desempeño.

¿Qué es el p-value?

Sea X una variable aleatoria que representa posibles conjuntos de prueba.
El conjunto observado es x.

Se define el p-value como:

$$\boxed{p = P\Big(\delta(X) \geq \delta(x) \mid H_0\Big)} \tag{4.47}$$

Es decir:

Suponiendo verdadera la hipótesis nula, ¿cuál es la probabilidad de observar una diferencia al menos tan grande como la obtenida?

- Un p-value pequeño aporta evidencia contra H_0.
- Un p-value grande indica que la diferencia no resulta suficientemente sorprendente bajo H_0.

Interpretación de la significancia

Elegimos un nivel de significancia α.
Valores habituales:

$$\alpha = 0{,}05 \qquad \text{o} \qquad \alpha = 0{,}01.$$

La regla de decisión es:

$$\boxed{p < \alpha \quad \Longrightarrow \quad \text{rechazar } H_0.}$$

Por ejemplo, si

$$p = 0{,}0047 < 0{,}01,$$

rechazamos H_0 al nivel de significancia del 1 %.

Cuidado

Un p-value de 0,0047 no significa que exista una probabilidad de 0,47 % de que H_0 sea verdadera.

Tampoco mide por sí solo la importancia práctica de la mejora.

¿Cómo calcular el p-value en NLP?

Algunas pruebas estadísticas tradicionales son:

- t-test: Un test paramétrico. Compara dos modelos y asume que los datos siguen una distribución normal.
- ANOVA (Análisis de Varianza). Compara las medias de tres o más grupos simultáneamente, asumiendo que los datos de cada grupo siguen una distribución normal y tienen varianzas similares.

Estas pruebas utilizan supuestos sobre la distribución de sus estadísticos.

En evaluación de NLP, tales supuestos no siempre son apropiados.

Alternativa: pruebas no paramétricas

Podemos generar numerosas versiones del experimento mediante procedimientos de muestreo.

Nos concentraremos en el **paired bootstrap test**.

¿Qué significa que una prueba sea pareada?

Una prueba es **pareada** cuando cada observación de un sistema corresponde a una observación del otro.

En NLP, ambos modelos se evalúan sobre los mismos documentos:

$$x = \{x_1, x_2, \ldots, x_n\}.$$

Documento	Modelo A	Modelo B
x_1	Correcto	Correcto
x_2	Correcto	Incorrecto
x_3	Incorrecto	Correcto
x_4	Correcto	Correcto

Idea fundamental

Cuando seleccionamos un documento, conservamos juntos su etiqueta verdadera y los resultados de ambos clasificadores.

El método bootstrap

El **bootstrap** consiste en obtener repetidamente muestras aleatorias **con reemplazo** a partir de una muestra original.

Supongamos que tenemos:

$$x = \{x_1, x_2, x_3, x_4, x_5\}.$$

Una posible muestra bootstrap es:

$$x^{(1)} = \{x_2, x_5, x_2, x_1, x_4\}.$$

Otra posible muestra es:

$$x^{(2)} = \{x_3, x_3, x_1, x_5, x_5\}.$$

- Cada muestra tiene tamaño $n = 5$.
- Un elemento puede aparecer varias veces.
- Algunos elementos pueden no aparecer.

Ejemplo: dos clasificadores y 10 documentos

Consideremos un conjunto de prueba con $n = 10$ documentos.
Cada documento puede pertenecer a uno de cuatro casos:

Caso	A	B
1	Correcto	Correcto
2	Correcto	Incorrecto
3	Incorrecto	Correcto
4	Incorrecto	Incorrecto

Ejemplo

	1	2	3	4	5	6	7	8	9	10	A%	B%	$\delta()$
x	AB	AB	AB	AB	AB	AB	AB	AB	AB	AB	.70	.50	.20
$x^{(1)}$	AB	AB	AB	AB	AB	AB	AB	AB	AB	AB	.60	.60	.00
$x^{(2)}$	AB	AB	AB	AB	AB	AB	AB	AB	AB	AB	.60	.70	-.10
...													
$x^{(b)}$													

$$\text{Accuracy}(A, x) = 0{,}70,$$

$$\text{Accuracy}(B, x) = 0{,}50.$$

Entonces:

$$\boxed{\delta(x) = 0{,}70 - 0{,}50 = 0{,}20.}$$

El modelo A supera a B por 20 puntos porcentuales en este test set.

Construcción de los test sets virtuales

Partimos de un conjunto original x con $n = 10$ documentos.

Para construir $x^{(1)}$:

1. Seleccionamos aleatoriamente un documento de x.
2. Copiamos su etiqueta y los resultados de ambos modelos.
3. Devolvemos el documento al conjunto.
4. Repetimos hasta obtener 10 selecciones.

Repetimos todo el proceso b veces:

$$x^{(1)}, x^{(2)}, \ldots, x^{(b)}.$$

Para cada muestra calculamos:

$$\delta(x^{(i)}) = M(A, x^{(i)}) - M(B, x^{(i)}).$$

Número de repeticiones

Se sugiere utilizar un número grande de muestras, por ejemplo $b = 10^5$.

¿Por qué debemos ajustar la diferencia?

Recordemos que observamos:

$$\delta(x) = 0{,}20.$$

Bajo H_0, queremos evaluar diferencias que serían sorprendentes si A no fuera mejor que B.

Lo natural sería contar cuantas veces $\delta(x^{(i)}) \geq 0$:

$$p = \frac{1}{b} \sum_{i=1}^{b} \mathbb{I}\left(\delta(x^{(i)}) - \delta(x) \geq 0\right).$$

Pero las muestras bootstrap se construyen a partir del conjunto original, donde A ya tiene una ventaja de 0,20.
Por ello, la distribución bootstrap de $\delta(x^{(i)})$ está aproximadamente centrada alrededor de $\delta(x)$, no alrededor de cero.

Consecuencia

No debemos tratar directamente la distribución bootstrap original como si fuera una distribución generada bajo H_0.

Necesitamos considerar las diferencias respecto de la ventaja observada.

Deducción del p-value bootstrap

La diferencia observada es $\delta(x)$.

Para cada muestra bootstrap calculamos la desviación respecto de esa diferencia:

$$\delta(x^{(i)}) - \delta(x).$$

Contamos cuándo esa desviación es al menos tan grande como $\delta(x)$:

$$\delta(x^{(i)}) - \delta(x) \geq \delta(x).$$

Sumando $\delta(x)$ en ambos lados:

$$\boxed{\delta(x^{(i)}) \geq 2\delta(x).}$$

Por tanto, el p-value empírico es:

$$\boxed{p = \frac{1}{b} \sum_{i=1}^{b} \mathbb{I}\left(\delta(x^{(i)}) \geq 2\delta(x)\right).}$$

Aquí $\mathbb{I}(\cdot)$ vale 1 si la condición se cumple y 0 en otro caso.

Ejemplo numérico del p-value

Supongamos que generamos:

$$b = 10\,000$$

muestras bootstrap.
La diferencia observada es:

$$\delta(x) = 0{,}20.$$

Según la ecuación (4.48), contamos las muestras que satisfacen:

$$\delta(x^{(i)}) \geq 2(0{,}20) = 0{,}40.$$

Supongamos que esto ocurre en solamente 47 muestras.

$$\boxed{p = \frac{47}{10\,000} = 0{,}0047.}$$

Como:

$$0{,}0047 < 0{,}01,$$

rechazamos H_0 al nivel del 1 %.

Significancia no es magnitud

Una diferencia puede ser

estadísticamente detectable

pero al mismo tiempo

muy pequeña en términos prácticos.

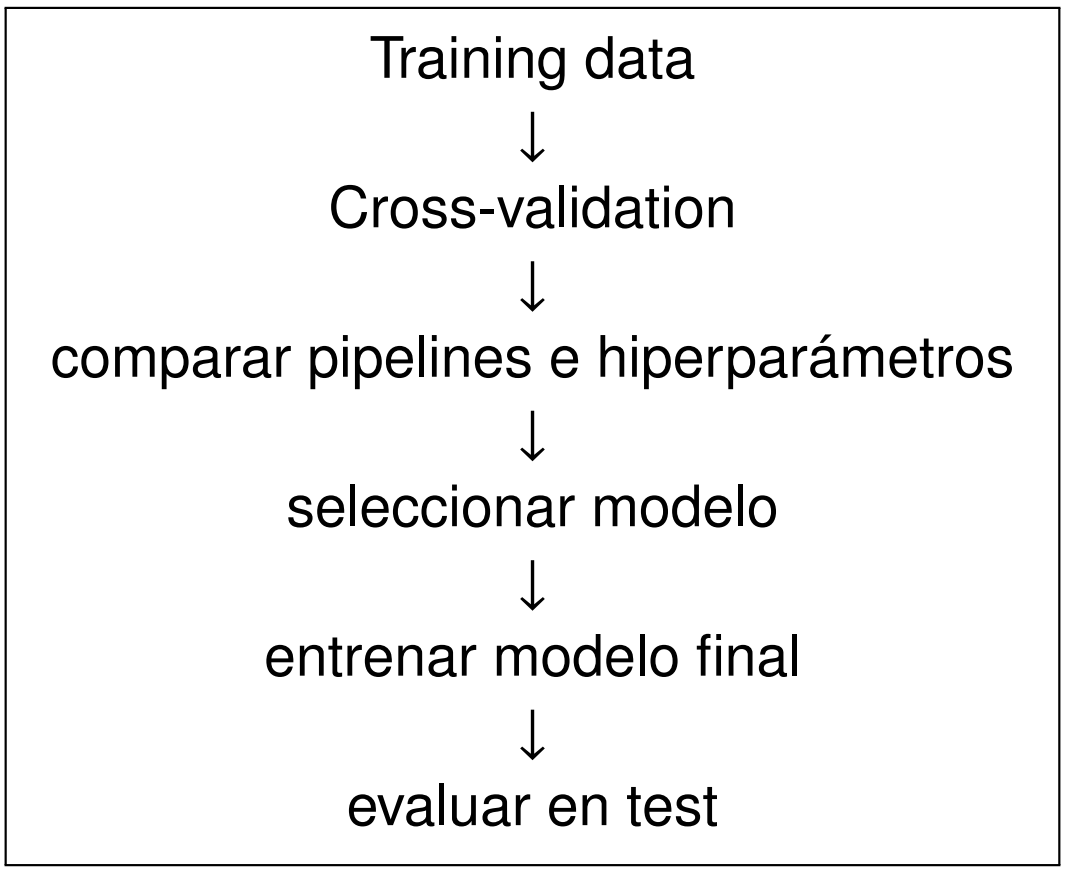
Por eso conviene reportar tanto

la magnitud de la diferencia

como evidencia sobre su estabilidad estadística.

Proceso completo de selección

Una estrategia correcta es:



Modelo final

Después de seleccionar la configuración

$$\theta^*,$$

podemos entrenar nuevamente utilizando **todos los datos de entrenamiento**:

$$\boxed{\text{Training completo} \xrightarrow{\theta^*} \text{modelo final.}}$$

Solo entonces evaluamos sobre

$$\boxed{\text{Test set.}}$$

Ejemplo

Queremos comparar:

$$M_1 = \text{CountVectorizer + LogisticRegression}$$

y

$$M_2 = \text{TfidfVectorizer + LogisticRegression.}$$

Podemos evaluar ambos mediante

$$\text{5-fold cross-validation}$$

con

$$F_{1,\text{macro}}$$

como métrica.

Después elegimos una configuración y reportamos su desempeño en el

$$\boxed{\text{test set final.}}$$

Comparación de modelos

Por ejemplo:

Modelo	**CV F_1 macro**	**Test F_1 macro**
Count + LR	0,79	–
TF-IDF + LR	0,83	–
TF-IDF + LR seleccionado	–	0,82

Durante selección utilizamos

solamente las métricas de validación.

El test se utiliza al final.

Las métricas no cuentan toda la historia

Después de evaluar un modelo conviene inspeccionar

sus errores.

Por ejemplo:

- falsos positivos;
- falsos negativos;
- clases que se confunden frecuentemente;
- documentos ambiguos;
- textos muy cortos o muy largos.

La matriz de confusión puede ayudar a detectar **patrones sistemáticos de error**.

Práctica #8: Clasificación y significancia estadística

Objetivo: Comparar dos representaciones de texto para clasificación multiclase y determinar si existe evidencia estadística de una diferencia de rendimiento.

Modelos:

- Modelo A: `TfidfVectorizer` + `LogisticRegression`.
- Modelo B: `CountVectorizer` + `LogisticRegression`.

Procedimiento:

1. Preparar los datos y construir los pipelines.
2. Evaluar mediante 5-fold cross-validation.
3. Comparar métricas multiclase.
4. Seleccionar hiperparámetros con GridSearchCV.
5. Evaluar ambos modelos en el test set.
6. Analizar errores de clasificación.
7. Aplicar el paired bootstrap test.

Pregunta central

¿Una diferencia observada de F_1 macro entre dos clasificadores representa evidencia suficiente para concluir que uno es mejor?

Paso 1: Cargar los datos

Utilizaremos cuatro categorías del conjunto *20 Newsgroups*.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_20newsgroups
from sklearn.model_selection import train_test_split

categorias = ["sci.space", "rec.sport.baseball", "comp.graphics", "talk.
    politics.mideast"]

datos = fetch_20newsgroups(subset="train", categories=categorias, remove
    =("headers", "footers", "quotes"), shuffle=True, random_state=42)

X = np.array(datos.data, dtype=object)
y = np.array(datos.target)

print("Documentos:", len(X))
print("Categorias:", datos.target_names)
```

Paso 2: Separar entrenamiento y prueba

Reservamos el 20 % de los datos para evaluar el desempeño final.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    stratify=y,
    random_state=42
)

print("Entrenamiento:", len(X_train))
print("Prueba:", len(X_test))
```

¿Dónde está el conjunto de validación?

Utilizaremos **5-fold cross-validation** dentro del conjunto de entrenamiento.
En cada iteración:

- 4 folds se utilizan para entrenar.
- 1 fold se utiliza para validar.

El test set permanece separado durante la selección de modelos e hiperparámetros.

Paso 3: Construir los pipelines

Comparamos dos representaciones manteniendo el mismo algoritmo de clasificación.

```
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import (
    CountVectorizer,
    TfidfVectorizer
)
from sklearn.linear_model import LogisticRegression

pipeline_count = Pipeline([
    ("vectorizer", CountVectorizer()),
    ("classifier", LogisticRegression(max_iter=2000))
])

pipeline_tfidf = Pipeline([
    ("vectorizer", TfidfVectorizer()),
    ("classifier", LogisticRegression(max_iter=2000))
])

modelos = {
    "CountVectorizer": pipeline_count,
    "TfidfVectorizer": pipeline_tfidf
}
```

Paso 4: Evaluación con 5-fold cross-validation

Evaluamos ambos pipelines utilizando las mismas particiones de validación cruzada.

```
from sklearn.model_selection import StratifiedKFold, cross_validate

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

metricas = {"Accuracy": "accuracy", "Precision macro": "precision_macro", "Recall macro": "
      recall_macro", "F1 macro": "f1_macro", "F1 micro": "f1_micro"}

resultados = []

for nombre, modelo in modelos.items():
    scores = cross_validate(modelo, X_train, y_train, cv=cv, scoring=metricas, n_jobs=-1)

    fila = {"Modelo": nombre}

    for metrica in metricas:
        valores = scores["test_" + metrica]
        fila[metrica] = valores.mean()
        fila[metrica + " std"] = valores.std()

    resultados.append(fila)

df_resultados = pd.DataFrame(resultados)
print(df_resultados.round(4).to_string(index=False))
```

Paso 5: Comparar F_1 macro y micro

```
comparacion = df_resultados.set_index("Modelo")[
    ["F1 macro", "F1 micro"]
]

print(comparacion.round(4))

comparacion.plot(
    kind="bar",
    figsize=(8, 4),
    rot=0
)

plt.ylabel("F1")
plt.ylim(0, 1)
plt.title("Comparacion de F1 macro y micro")
plt.tight_layout()
plt.show()
```

Pregunta

¿Por qué en este problema de clasificación multiclase de etiqueta única se cumple $F_{1,micro} = Accuracy$?

Paso 6: Selección de hiperparámetros

Utilizaremos `GridSearchCV` para explorar diferentes configuraciones de cada pipeline.

```
from sklearn.model_selection import GridSearchCV

param_grid = {
    "vectorizer__ngram_range": [(1, 1), (1, 2)],
    "vectorizer__min_df": [1, 3],
    "classifier__C": [0.1, 1, 10]
}

busquedas = {}
for nombre, modelo in modelos.items():
    grid = GridSearchCV(
        estimator=modelo,
        param_grid=param_grid,
        scoring="f1_macro",
        cv=cv,
        n_jobs=-1,
        refit=True
    )

    grid.fit(X_train, y_train)
    busquedas[nombre] = grid

    print(nombre)
    print(grid.best_params_)
    print("Mejor F1 macro:", grid.best_score_)
```

Paso 7: Comparar las mejores configuraciones

Comparamos los mejores resultados de validación cruzada de ambos pipelines.

```
resumen = []

for nombre, grid in busquedas.items():
    resumen.append({
        "Modelo": nombre,
        "F1 macro CV": grid.best_score_,
        "Parametros": grid.best_params_
    })

df_resumen = pd.DataFrame(resumen)
print(df_resumen.to_string(index=False))

mejor_nombre = max(busquedas,key=lambda nombre: busquedas[nombre].best_score_)

print("Modelo seleccionado:", mejor_nombre)
```

Importante

La selección se realiza utilizando **únicamente el conjunto de entrenamiento**.
Con `refit=True`, cada mejor configuración se reentrena automáticamente con todos los datos de entrenamiento.

Conservaremos ambos modelos optimizados para realizar posteriormente el paired bootstrap test.

Paso 8: Evaluar los modelos en el test set

Obtenemos las predicciones de los dos modelos sobre exactamente los mismos documentos.

```
from sklearn.metrics import f1_score, classification_report

modelo_count = busquedas["CountVectorizer"].best_estimator_
modelo_tfidf = busquedas["TfidfVectorizer"].best_estimator_

pred_count = modelo_count.predict(X_test)
pred_tfidf = modelo_tfidf.predict(X_test)

f1_count = f1_score(y_test, pred_count, average="macro")
f1_tfidf = f1_score(y_test, pred_tfidf, average="macro")

print("F1 Count:", f1_count)
print("F1 TF-IDF:", f1_tfidf)

print(classification_report(
    y_test, pred_tfidf,
    target_names=datos.target_names,
    zero_division=0
))
```

Paso 9: Matriz de confusión

Visualizamos qué categorías se confunden con mayor frecuencia.

```
from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_predictions(
    y_test,
    pred_tfidf,
    display_labels=datos.target_names,
    xticks_rotation=45,
    cmap="Blues"
)

plt.title("Matriz de confusion: TF-IDF")
plt.tight_layout()
plt.show()
```

Preguntas

- ¿Qué clases presentan más errores?
- ¿Existen clases con vocabulario similar?
- ¿La matriz de confusión permite identificar diferencias entre Precision y Recall?

Paso 10: Analizar errores de clasificación

Inspeccionamos documentos cuya clase predicha es diferente de la clase verdadera.

```
errores = np.where(y_test != pred_tfidf)[0]

print("Numero de errores:", len(errores))

for i in errores[:5]:
    print("=" * 60)

    print("Texto:")
    print(X_test[i][:500])

    print("Clase real:")
    print(datos.target_names[y_test[i]])

    print("Clase predicha:")
    print(datos.target_names[pred_tfidf[i]])
```

¿Qué factores podrían explicar los errores?

- Textos demasiado cortos.
- Vocabulario compartido entre categorías.
- Documentos ambiguos.
- Información insuficiente.

Práctica #8 – Entregable

Se definirá próxima clase...

Resumen

La evaluación de clasificadores de texto requiere:

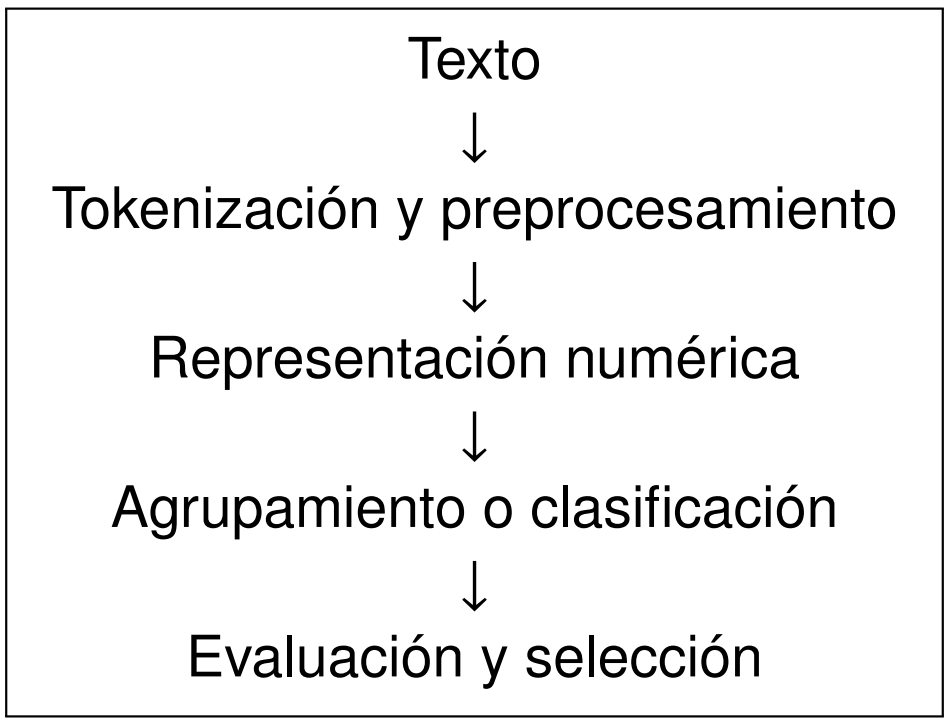
$$\boxed{\text{Métricas} + \text{Validación} + \text{Selección}}$$

En particular:

- Accuracy puede ser engañosa con clases desbalanceadas;
- Precision y Recall describen errores diferentes;
- F_1 combina ambas métricas;
- macro y micro averaging resumen problemas multiclase;
- cross-validation permite comparar configuraciones;
- el test set debe reservarse para la evaluación final;
- la regularización ayuda a controlar overfitting.

Tema 4: Aprendizaje de máquina para textos

La secuencia completa ha sido:



el texto se convierte en un problema estándar de aprendizaje de máquina.

¡Gracias!